

I2C Driver Documentation

1. Driver Overview

I²C, or **Inter-Integrated Circuit**, is a synchronous, two-wire serial communication protocol. An I²C driver is a software module that allows the STM32F401RE microcontroller to communicate with external devices using the I²C protocol. The STM32F401RE supports I²C communication through dedicated peripheral registers and alternate-function GPIO pins.

Driver Responsibilities

The I²C driver is responsible for:

- Enabling the GPIO and I²C peripheral clocks.
- Configuring the SCL and SDA pins.
- Setting the I²C communication speed.
- Generating START and STOP conditions.
- Sending the slave address.
- Transmitting data.
- Receiving data.
- Managing ACK and NACK signals.
- Checking communication status.
- Handling errors and timeouts.

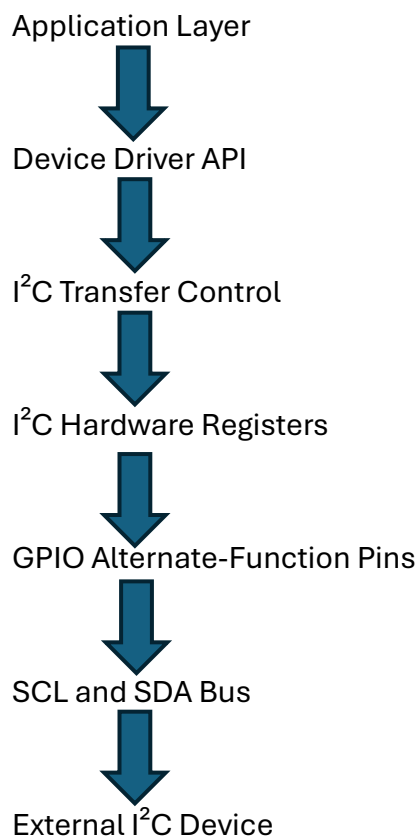
The STM32F401RE can operate as an I²C master or slave. In master mode, it controls the communication and generates the clock signal.

2. I²C Communication

An I²C communication sequence consists of:

1. START condition.
2. Slave address.
3. Read or write bit.
4. Data transfer.
5. ACK or NACK response.
6. STOP condition.

3. Driver Architecture



4. Initialization Process

- Enable the GPIO and I²C peripheral clocks.
- Configure the SCL and SDA pins for the correct alternate function.
- Set both pins to open-drain mode with suitable pull-up resistors.
- Disable the I²C peripheral before configuration.
- Configure the APB1 clock, I²C speed, and rise-time settings.
- Enable acknowledgement and activate the I²C peripheral.
- Check that the bus is free before starting communication.

5. I2C Communication Flow

Master Transmission

The driver performs the following sequence:

1. Generate a START condition.
2. Wait for the SB flag.
3. Send the slave address with the write bit.
4. Wait for the ADDR flag.
5. Clear the ADDR flag by reading SR1 followed by SR2.
6. Wait for the TXE flag and send the data bytes.
7. Wait for the BTF flag after the final byte.
8. Generate a STOP condition.

Master Reception

The driver performs the following sequence:

1. Generate a START condition.
2. Wait for the SB flag.
3. Send the slave address with the read bit.
4. Wait for the ADDR flag.

5. Configure ACK for the received bytes.
6. Clear the ADDR flag by reading SR1 followed by SR2.
7. Wait for the RXNE flag and read the data bytes.
8. Disable ACK and generate a NACK before the final byte.
9. Generate a STOP condition.
10. Restore the ACK configuration.

6. Blocking Communication APIs

These APIs use polling and wait until communication completes.

I2C_MasterSendData()

Purpose: Transmit a complete data buffer to a slave device.

Internal Operation

Uses:

- SB flag
- ADDR flag
- TXE flag
- BTF flag
- Data Register (DR)

Abstraction: Provides a single API for complete data transmission without requiring the application to monitor hardware flags.

I2C_MasterReceiveData()

Purpose: Receive a complete data buffer from a slave device.

Internal Operation

Uses:

- SB flag

- ADDR flag
- RXNE flag
- ACK/NACK management

Abstraction: Provides complete receive functionality while hiding reception timing and ACK control details.

I2C_SlaveSendData()

Purpose: Transmit one byte from slave device.

Abstraction: Application only provides data while driver handles register access.

I2C_SlaveReceiveData()

Purpose: Read one byte received by slave device.

Abstraction: Returns received data without exposing register operations.

7. Interrupt Driven Communication

The driver supports non-blocking communication using interrupts.

I2C_MasterSendDataIT()

Purpose: Start interrupt-driven transmission.

Internal Actions

- Stores transmit buffer information.
- Marks peripheral busy.
- Generates START condition.
- Enables event, buffer and error interrupts.

Abstraction: Application starts transmission using a single API while the driver handles data transfer in interrupt context.

I2C_MasterReceiveDataIT()

Purpose: Start interrupt-driven reception.

Internal Actions

- Stores receive buffer information.
- Marks peripheral busy.
- Generates START condition.
- Enables interrupt sources.

Abstraction: Application does not need to manage receive timing or hardware events.

8. Interrupt Event Handling

I2C_EV_IRQHandling()

This function acts as the central event interrupt handler.

The handler processes:

SB Event: Triggered after START generation.

Action:

- Sends slave address.

ADDR Event: Triggered after address transmission.

Action:

- Clears ADDR flag.

BTF Event: Triggered when byte transfer completes.

Action:

- Finishes transmission.
- Generates STOP condition.
- Notifies application.

STOPF Event: Triggered in slave mode when STOP condition is detected.

Action:

- Clears STOP flag.
- Notifies application.

TXE Event: Triggered when transmit register becomes empty.

Action:

- Loads next byte into DR.

RXNE Event: Triggered when data becomes available.

Action:

- Reads data from DR.
- Handles final-byte reception.
- Generates STOP condition.
- Notifies application.

9. Error Interrupt Handling

I2C_ER_IRQHandling()

Handles I2C communication errors.

Bus Error (BERR): Occurs due to invalid START or STOP conditions.

Driver Actions:

- Clears error flag.
- Generates STOP.
- Closes active transaction.
- Notifies application.

Arbitration Lost (ARLO): Occurs in multi-master systems.

Driver Actions:

- Clears flag.

- Closes transaction.
- Notifies application.

Acknowledge Failure (AF): Occurs when receiver does not acknowledge transmitted data.

Driver Actions:

- Clears flag.
- Generates STOP.
- Closes transaction.
- Notifies application.

Overrun / Underrun (OVR): Occurs when data is not read or written in time.

Driver Actions:

- Clears flag.
- Closes transaction.
- Notifies application.

Timeout Error: Occurs mainly in SMBus operation.

Driver Actions:

- Clears flag.
- Closes transaction.
- Notifies application.

10. Peripheral Control APIs

I2C PeripheralControl()

Purpose: Enable or disable the I2C peripheral.

Internal Register: CR1.PE

Abstraction: Provides peripheral control without direct register access.

I2C PericlockControl()

Purpose: Enable or disable I2C clock.

Internal Register: RCC APB1ENR

Abstraction: Hides RCC register manipulation.

I2C DeInit()

Purpose: Reset I2C peripheral.

Internal Register: RCC APB1RSTR

Abstraction: Provides simple peripheral reset functionality.

I2C ManageAcking()

Purpose: Enable or disable ACK generation.

Internal Register: CR1.ACK

Abstraction: Provides ACK control through a dedicated API.

11. Interrupt Configuration APIs

I2C IRQInterruptConfig()

Purpose: Enable or disable NVIC interrupts.

Internal Registers

- ISER
- ICER

Abstraction Hides NVIC register programming.

I2C IRQPriorityConfig()

Purpose: Configure interrupt priority.

Internal Registers

- IPR registers

Abstraction: Provides simple interrupt priority configuration.

12. Internal Driver Functions

The following functions are internal helper functions and are not intended for direct application use:

- I2C_GenerateStartCondition()
- I2C_GenerateStopCondition()
- I2C_ExecuteAddressPhaseWrite()
- I2C_ExecuteAddressPhaseRead()
- I2C_ClearADDRFlag()
- I2C_MasterHandleTXEInterrupt()
- I2C_MasterHandleRXNEInterrupt()
- I2C_CloseSendData()
- I2C_CloseReceiveData()

These functions implement low-level protocol operations and support the public APIs.

13. Abstraction Provided by the Driver

The driver hides the following hardware details:

- RCC clock control registers
- I2C timing calculations
- START and STOP generation
- Address phase handling
- ACK/NACK management
- Status flag monitoring
- Interrupt enable registers
- NVIC configuration
- Error recovery mechanisms

The application only interacts through high-level APIs and callback events.

14. Conclusion

The I2C driver provides a complete abstraction layer for the STM32F401 I2C peripheral. It supports both polling-based and interrupt-driven communication while handling protocol details such as address transmission, ACK management, interrupt servicing, and error recovery internally. This approach improves code modularity, maintainability, and reusability while keeping the application independent of hardware register operations.